

EVALUACIÓN	Obligatorio 1	GRUPO		FECHA	22/10/2026
MATERIA	Estructuras de Datos y Algoritmos 2				
CARRERA	Ingeniería en Sistemas				
CONDICIONES	- Puntaje máximo: 15 puntos - Puntaje mínimo: 0 puntos - Fecha de entrega: 22/10/2026 hasta las 21:00 horas en gestion.ort.edu.uy (max. 40Mb en formato zip) Uso de material de apoyo y/o consulta <u>Inteligencia Artificial Generativa</u> - Seguir las pautas de los docentes: Se deben seguir las instrucciones específicas de los docentes sobre cómo utilizar la IA en cada curso. - Citar correctamente las fuentes y usos de IA: Siempre que se utilice una herramienta de IA para generar contenido, se debe citar adecuadamente la fuente y la forma en que se utilizó. - Se debe indicar: a. la /las herramientas utilizadas y b. el contexto de uso: por ejemplo, generación de ideas, redacción inicial, análisis de datos, corrección de estilo, etc. - Todo contenido producido por IAG debe ser revisado y verificado; cualquier error presente será responsabilidad del estudiante. - Ser responsables con el uso de la IA: Conocer los riesgos y desafíos, como la creación de “alucinaciones”, los peligros para la privacidad, las cuestiones de propiedad intelectual, los sesgos inherentes y la producción de contenido falso. - El uso de herramientas de Inteligencia Artificial Generativa (IAG) puede emplearse como apoyo en el proceso de aprendizaje. Sin embargo, no sustituye el razonamiento crítico ni la elaboración personal. El estudiante es responsable de que el trabajo presentado refleje su propia comprensión, análisis y proceso cognitivo sobre el tema. - La defensa es obligatoria y eliminatoria. - La defensa será presencial salvo en casos en que se habilite la defensa oral sincrónica en forma remota. Defensa Fecha de defensa: Clase siguiente a la entrega del 22/10/2026 <u>La defensa es obligatoria y eliminatoria.</u> El docente es quien definirá y comunicará la modalidad, y mecánica de defensa. <u>La no presentación a la misma implica la pérdida de la totalidad de los puntos del Obligatorio.</u> IMPORTANTE: 1. Inscribirse 2. Formar grupos de hasta 2 personas del mismo dictado 3. Subir el trabajo a Gestión antes de la hora indicada (ver hoja al final del documento: “RECORDATORIO”) Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con la Coordinadora de cursos o Coordinación Adjunta antes de las 20:00h del día de la entrega, a través del mail: adjuntos_ei@ort.edu.uy.				

Obligatorio 1

Ejercicio 1 - Catálogo del museo

Descripción

El Museo Nacional conserva dos colecciones que nunca compartieron un sistema de identificación. Las **monedas** se identifican por su número de catálogo, que es un entero. Las **pinturas** se identifican por su título, que es una cadena.

El área de conservación necesita un sistema que registre altas y responda consultas sobre las dos colecciones. Una pieza que ya está registrada no se vuelve a registrar. Ninguna pieza se da de baja. Las dos colecciones son independientes: una moneda nunca se compara con una pintura.

Cada colección se mantiene ordenada según su propio criterio:

1. Las **monedas** se ordenan por número de catálogo, de menor a mayor.
2. Las **pinturas** se ordenan por título, en orden alfabético.

El sistema procesa una secuencia de operaciones. Para cada consulta debe **imprimir el resultado en el orden de la colección consultada**.

Entrada

- La primera línea contiene un entero N ($1 \leq N \leq 2 \times 10^5$), la cantidad de operaciones.
- Las siguientes N líneas contienen una operación cada una. Toda operación empieza por su nombre, seguido de la letra de la colección: **M** para monedas y **P** para pinturas.

Las operaciones de alta registran una pieza. Si la pieza ya estaba registrada, la operación no tiene efecto.

```
ALTA M <C>
ALTA P <T>
```

Las operaciones de búsqueda consultan si una pieza está registrada.

```
BUSCAR M <C>
BUSCAR P <T>
```

Las operaciones de rango consultan un intervalo. El primer valor es el extremo inferior y el segundo es el extremo superior.

```
RANGO M <desde> <hasta>
RANGO P <desde> <hasta>
```

El intervalo de **RANGO** es **cerrado en los dos extremos**. Se reporta toda pieza registrada cuya clave sea **mayor o igual** que **desde** y **menor o igual** que **hasta**. Una pieza cuya clave coincide exactamente con **desde** se reporta. Una pieza cuya clave coincide exactamente con **hasta** también se reporta.

Donde:

- C : número de catálogo de una moneda, con $1 \leq C \leq 10_{12}$.
- T : título de una pintura. Cadena de 1 a 20 letras latinas minúsculas, sin espacios.

Los valores `desde` y `hasta` tienen el mismo formato y las mismas cotas que la clave de la colección consultada. No hace falta que estén registrados.

Además:

- En toda operación `RANGO`, el extremo inferior es menor o igual que el extremo superior, según el orden de la colección consultada.
- La suma de las piezas reportadas por todas las operaciones `RANGO` no supera 10_6 .

Salida

- `ALTA` no imprime nada, tampoco cuando la pieza ya estaba registrada.
- `BUSCAR` imprime `si` si la pieza está registrada, o `no` en caso contrario.
- `RANGO` imprime una línea por cada pieza registrada que pertenece al intervalo cerrado, en el orden de la colección. Los dos extremos se incluyen. Si ninguna pieza pertenece al intervalo, no imprime nada.

Cada pieza se imprime así:

- Una **moneda**: su número de catálogo.
- Una **pintura**: su título.

Restricciones

- Implementar un **único TAD árbol AVL parametrizado por tipo** (`template` en C++, genérico en Java) e instanciarlo una vez por colección. No se acepta una implementación distinta por colección.
- `ALTA` y `BUSCAR` en orden temporal $O(\log K)$ en el peor caso, siendo K la cantidad de piezas registradas en esa colección al momento de ejecutar la operación.
- `RANGO` en orden temporal $O(\log K + R)$ en el peor caso, siendo R la cantidad de piezas que esa consulta reporta.

Ejemplo

Input 1

```
12
ALTA M 500
ALTA M 120
ALTA M 900
ALTA M 500
ALTA P girasoles
ALTA P guernica
ALTA P gioconda
BUSCAR M 120
BUSCAR M 700
RANGO M 100 600
RANGO M 1000 2000
RANGO P a z
```

Output 1

```
si
no
120
500
gioconda
girasoles
guernica
```

Explicación 1

El cuarto `ALTA M 500` no tiene efecto, porque el catálogo 500 ya estaba registrado. La colección de monedas queda con 120, 500 y 900.

`BUSCAR M 120` encuentra la moneda. `BUSCAR M 700` no la encuentra, porque nunca se registró.

`RANGO M 100 600` reporta 120 y 500. La moneda 900 supera el extremo superior.

`RANGO M 1000 2000` no reporta nada. Ninguna moneda registrada llega al extremo inferior.

`RANGO P a z` reporta las tres pinturas, en orden alfabético. Notar que se imprimen ordenadas, y no en el orden en que se dieron de alta.

Input 2

```
10
ALTA M 10
ALTA M 20
ALTA M 30
ALTA M 40
RANGO M 20 30
RANGO M 10 40
RANGO M 21 29
RANGO M 20 20
BUSCAR M 20
RANGO M 41 50
```

Output 2

```
20
30
10
20
30
40
20
si
```

Explicación 2

Este ejemplo muestra que los dos extremos del intervalo se incluyen.

Consulta	Reporta	Motivo
RANGO M 20 30	20 y 30	Las dos monedas coinciden con un extremo, y las dos entran.
RANGO M 10 40	10, 20, 30 y 40	El intervalo abarca la colección entera.
RANGO M 21 29	nada	Entre 21 y 29 no hay ninguna moneda registrada.
RANGO M 20 20	20	Los dos extremos coinciden, así que el intervalo contiene una sola clave.
RANGO M 41 50	nada	Toda la colección queda por debajo del extremo inferior.

Comparar la primera consulta con la tercera deja clara la diferencia. `RANGO M 20 30` reporta las monedas 20 y 30 porque los extremos entran. `RANGO M 21 29` no reporta ninguna, porque al mover los extremos un lugar hacia adentro las dos quedan afuera.

Input 3

```
9
ALTA P guernica
ALTA P girasoles
ALTA P gioconda
ALTA P nenufares
ALTA P grito
RANGO P gi gz
RANGO P a f
BUSCAR P gioconda
RANGO P grito nenufares
```

Output 3

```
gioconda
girasoles
grito
guernica
si
grito
guernica
nenufares
```

Explicación 3

Las cinco pinturas, en orden alfabético, quedan así:

```
gioconda
girasoles
grito
guernica
nenufares
```

RANGO P gi gz reporta las cuatro pinturas que empiezan con g . El extremo inferior gi es anterior a gioconda , porque es un prefijo más corto. El extremo superior gz es posterior a guernica , porque comparando letra a letra la z es posterior a la u . La pintura nenufares queda afuera, porque la n es posterior a la g .

RANGO P a f no reporta nada. Todas las pinturas empiezan con g o con n , y las dos letras son posteriores a la f .

RANGO P grito nenufares reporta tres pinturas. Los dos extremos están registrados y los dos se incluyen: grito abre el intervalo y nenufares lo cierra.

Input 4

```
5
BUSCAR M 1
RANGO P a z
ALTA P unica
RANGO P unica unica
BUSCAR P otra
```

Output 4

```
no
unica
no
```

Explicación 4

Las dos primeras operaciones consultan colecciones vacías. `BUSCAR M 1` imprime `no`. `RANGO P a z` no imprime nada, porque no hay ninguna pintura registrada.

`RANGO P unica unica` consulta un intervalo cuyos dos extremos coinciden. La pintura `unica` está registrada y coincide con los dos extremos, así que se reporta.

`BUSCAR P otra` imprime `no`. La colección de pinturas tiene una sola pieza, y no es esa.

Ejercicio 2 - El taller de imprenta

Descripción

Un taller de imprenta compone palabras con tipos móviles: piezas de metal, una por letra. Para componer una palabra, el operario necesita un tipo por cada letra que la palabra usa. Las letras repetidas necesitan tipos repetidos.

El taller guarda los tipos agrupados en cajones. Dos palabras se componen con el mismo cajón cuando necesitan exactamente los mismos tipos, es decir, las mismas letras y cada una la misma cantidad de veces.

Por ejemplo, `roma`, `amor` y `mora` se componen con el mismo cajón, porque las tres necesitan un tipo de cada una de las letras `a`, `m`, `o` y `r`. En cambio `ala` y `lea` se componen con cajones distintos: `ala` necesita dos tipos de la letra `a`, y `lea` necesita uno solo más un tipo de la letra `e`.

El taller registra las palabras del catálogo de la temporada. Después recibe consultas. Para cada palabra consultada hay que informar **cuántas palabras registradas se componen con el mismo cajón que ella**.

Dos reglas que conviene tener presentes:

1. Dos palabras registradas iguales cuentan como **dos palabras distintas**.

2. Consultar una palabra **no la registra**. Una palabra consultada se cuenta a sí misma solo si además fue registrada.

Al terminar las consultas hay que informar cuántos cajones distintos hicieron falta para las palabras registradas, y cuántas palabras se componen con el **cajón más grande**. El cajón más grande es el que compone **más palabras registradas**. La cantidad de tipos que necesita un cajón no interviene en ese criterio. Si varios cajones empatan, la cantidad a informar es la misma para cualquiera de ellos.

Entrada

- La primera línea contiene un entero N ($1 \leq N \leq 10^5$), la cantidad de palabras registradas.
- Las siguientes N líneas contienen una palabra registrada cada una.
- La línea siguiente contiene un entero Q ($1 \leq Q \leq 10^5$), la cantidad de consultas.
- Las siguientes Q líneas contienen una palabra consultada cada una.

Toda palabra, registrada o consultada, es una cadena de 1 a 20 letras latinas minúsculas, sin espacios.

Salida

- Imprima Q líneas, una por consulta y en el orden en que llegan. La línea i contiene la cantidad de palabras registradas que se componen con el mismo cajón que la consulta i . Si ninguna palabra registrada usa ese cajón, imprima `0`.
- Imprima una última línea con dos enteros separados por un espacio: la cantidad de cajones distintos que hicieron falta para las N palabras registradas, y la cantidad de palabras que se componen con el cajón más grande, es decir, la mayor cantidad de palabras registradas que comparten un mismo cajón.

La salida tiene siempre $Q+1$ líneas.

Restricciones

- Utilizar una **tabla de hash abierta**, con resolución de colisiones por encadenamiento.
- Registrar una palabra y responder una consulta, las dos en orden temporal $O(L)$ promedio, siendo L el largo de la palabra involucrada. La cota cubre el **proceso completo** de cada operación: obtener a partir de la palabra la clave que identifica su cajón, calcular la función de hash sobre esa clave, y hacer la operación sobre la tabla, incluidas las comparaciones dentro del bucket. El orden es exactamente ese: **no admite ningún término que dependa de la cantidad de palabras que comparten cajón**, que puede llegar a ser N .
- Se espera una **decisión sobre el factor de carga**: dimensionar la tabla de modo que el factor de carga se mantenga sano durante todo el ejercicio, sin necesidad de hacer rehash y sin ocupar memoria de más.
- Obtener la última línea de la salida en $O(1)$ en el peor caso. Los dos valores que pide, cajones distintos y palabras del cajón más grande, se mantienen actualizados durante el registro, con $O(1)$ extra por palabra. No admite recorrer la tabla ni las N palabras al terminar, ni ordenar los cajones.
- Resolver el problema completo en orden temporal lineal, en promedio, respecto del tamaño total de la entrada.

El alfabeto tiene 26 letras y se considera constante a efectos del orden.

Ejemplo

Input 1

```
6
roma
amor
mora
sol
los
casa
3
ramo
sol
perro
```

Output 1

```
3
2
0
3 3
```

Explicación 1

Las seis palabras registradas se reparten en tres cajones:

Cajón	Tipos que necesita	Palabras registradas
1	a m o r	roma, amor, mora
2	l o s	sol, los
3	a a c s	casa

ramo necesita los tipos a , m , o y r , así que usa el cajón 1. Ese cajón tiene 3 palabras registradas.

sol usa el cajón 2, que tiene 2 palabras registradas. Notar que sol está registrada y también se consulta: se cuenta a sí misma.

perro necesita los tipos e , o , p , r y r . Ninguna palabra registrada usa ese cajón, así que la respuesta es 0 .

La última línea informa que hicieron falta 3 cajones, y que el cajón más grande compone 3 palabras. Notar que el cajón 3 necesita tantos tipos como el cajón 1, pero compone una sola palabra: el tamaño de un cajón es la cantidad de palabras que compone, no la cantidad de tipos que necesita.

Input 2

```
5
ala
ala
lea
ala
eal
2
ala
ale
```

Output 2

```
3
2
2 3
```

Explicación 2

Este ejemplo muestra las dos reglas del enunciado.

Las cinco palabras registradas usan dos cajones:

Cajón	Tipos que necesita	Palabras registradas
1	a a l	ala, ala, ala
2	a e l	lea, eal

La palabra `ala` está registrada tres veces. **Las tres cuentan por separado**, así que su cajón compone 3 palabras.

Los dos cajones son distintos aunque compartan las letras `a` y `l`. El primero necesita **dos** tipos de la letra `a`, y el segundo necesita **uno solo** más un tipo de la letra `e`. La cantidad de cada letra importa, no solo qué letras aparecen.

La consulta `ale` usa el cajón 2, que compone 2 palabras. Notar que `ale` no está registrada, y consultarla tampoco la registra: la última línea sigue informando 5 palabras repartidas en 2 cajones, con 3 en el mayor.

Input 3

```
1
z
3
z
zz
a
```

Output 3

```
1
0
0
1 1
```

Explicación 3

Hay una sola palabra registrada, así que hay un solo cajón y ese cajón compone una sola palabra.

`z` usa ese cajón y la respuesta es `1`.

`zz` necesita **dos** tipos de la letra `z`, así que usa un cajón distinto del de `z`. Dos palabras de largos distintos nunca se componen con el mismo cajón, porque necesitan cantidades distintas de tipos.

`a` necesita un tipo de la letra `a`, que ninguna palabra registrada usa.

Input 4

```
4
abc
bca
cab
acb
2
bac
abd
```

Output 4

```
4
0
1 4
```

Explicación 4

Las cuatro palabras registradas son reordenamientos de las mismas tres letras, así que las cuatro se componen con el mismo cajón. Hizo falta un solo cajón, y ese cajón compone las 4 palabras.

`bac` usa ese mismo cajón, así que la respuesta es `4`.

`abd` necesita un tipo de la letra `d`, que ese cajón no tiene. Ninguna palabra registrada usa el cajón de `abd`, así que la respuesta es `0`.

Ejercicio 3 - Consolidación de respaldos

Descripción

Un servidor guarda N archivos de respaldo. Para liberar espacio, el operario los consolida hasta que queda un único archivo.

Consolidar dos archivos obliga a leer los dos y escribir el resultado, así que **cuesta la suma de sus tamaños**. El operario repite siempre el mismo paso:

1. Elegir los **dos archivos más chicos** que estén disponibles en ese momento.
2. Consolidarlos en un archivo nuevo, cuyo tamaño es la **suma** de los tamaños de los dos.
3. El costo de esa consolidación es el tamaño del archivo nuevo.
4. Los dos archivos consolidados dejan de estar disponibles. **El archivo nuevo queda disponible**, y más adelante puede volver a consolidarse con otro.

El paso se repite hasta que queda un solo archivo disponible.

Determine el **costo total**, que es la suma de los costos de todas las consolidaciones realizadas.

Si en algún momento hay más de dos archivos con el tamaño mínimo, no importa cuáles dos se elijan: el costo total es el mismo.

Entrada

- La primera línea contiene un entero N ($1 \leq N \leq 2 \times 10_6$), la cantidad de archivos de respaldo.
- Las siguientes N líneas contienen un entero T ($1 \leq T \leq 10_6$) cada una, el tamaño de un archivo.

Salida

Imprima un único entero: el costo total de consolidar los N archivos en uno solo.

Si hay un único archivo, no se realiza ninguna consolidación y el costo total es `0`.

Restricciones

- Utilizar un **heap binario** para resolver el problema.
- Resolver en orden temporal $O(N \log N)$ en el peor caso, siendo N la cantidad de archivos.
- Orden espacial $O(N)$.
- El costo total puede superar el rango de un entero de 32 bits.

Ejemplo

Input 1

4
1
2
3
4

Output 1

19

Explicación 1

Paso	Se consolidan	Costo	Archivos disponibles al terminar
1	1 y 2	3	3, 3, 4
2	3 y 3	6	6, 4
3	4 y 6	10	10

El costo total es $3+6+10=19$.

Notar el paso 2. De los dos archivos de tamaño 3, uno es el archivo original y el otro es el que salió del paso 1. Los archivos originales de tamaño 1 y 2 ya no están disponibles, pero su contenido sigue adentro del archivo nuevo. Consolidar el archivo nuevo vuelve a leer y a escribir ese contenido, y por eso se paga de nuevo.

Los tamaños de los archivos suman $1+2+3+4=10$, pero el costo total es 19. Cada archivo original se paga una vez por cada consolidación que copia su contenido, ya sea directamente o adentro de un archivo nuevo que lo contiene:

$$1 \times 3 + 2 \times 3 + 3 \times 2 + 4 \times 1 = 19$$

El contenido del archivo de tamaño 1 se copia en las tres consolidaciones: en el paso 1 como archivo original, y en los pasos 2 y 3 adentro de los archivos nuevos de tamaño 3 y 6. El contenido del archivo de tamaño 4 se copia solo en la última.

Input 2

4
4
5
6
7

Output 2

44

Explicación 2

Paso	Se consolidan	Costo	Archivos disponibles al terminar
1	4 y 5	9	6, 7, 9
2	6 y 7	13	9, 13
3	9 y 13	22	22

El costo total es $9+13+22=44$.

Este ejemplo muestra que **hay que volver a buscar los dos más chicos en cada paso**. En el paso 2 el archivo nuevo de tamaño 9 ya no es de los dos más chicos: los más chicos son el 6 y el 7, que todavía no se tocaron.

Recorrer los archivos de menor a mayor una sola vez y consolidarlos en ese orden da un resultado distinto y equivocado:

Paso	Se consolidan	Costo
1	4 y 5	9
2	9 y 6	15
3	15 y 7	22

Eso da 46, no 44. El error está en el paso 2: consolida el archivo nuevo cuando todavía había archivos más chicos disponibles.

Input 3

1
42

Output 3

0

Explicación 3

Hay un solo archivo, así que no hay nada que consolidar. No se realiza ninguna consolidación y el costo total es 0.

El costo total **no** es 42. El tamaño de un archivo se paga cuando una consolidación copia su contenido, y acá no hay ninguna consolidación.

Input 4

2
7
13

Output 4

20

Explicación 4

Con dos archivos hay una sola consolidación, que cuesta $7+13=20$.

Este es el único caso en el que el costo total coincide con la suma de los tamaños. Con dos archivos cada uno participa exactamente de una consolidación, así que cada tamaño se paga una sola vez.

Input 5

6
1
1
1
1
1
1
1

Output 5

16

Explicación 5

Paso	Se consolidan	Costo	Archivos disponibles al terminar
1	1 y 1	2	1, 1, 1, 1, 2
2	1 y 1	2	1, 1, 2, 2
3	1 y 1	2	2, 2, 2
4	2 y 2	4	2, 4
5	2 y 4	6	6

El costo total es $2+2+2+4+6=16$, contra una suma de tamaños de apenas 6.

Los tres primeros pasos consolidan los seis archivos originales de a pares, porque mientras queden archivos de tamaño 1 esos son los más chicos. Recién en el paso 4 se empiezan a consolidar los archivos nuevos entre sí.

Consolidar en el orden de la entrada, uniendo cada archivo al resultado anterior, daría $2+3+4+5+6=20$. La diferencia es grande justamente porque hay muchos archivos del mismo tamaño.

Ejercicio 4 - Orden de compilación

Descripción

Una herramienta de compilación procesa los V módulos de un proyecto, numerados de 1 a V . Cada módulo tiene además una **prioridad**, y un módulo puede depender de otros: no se puede compilar hasta que todos los módulos de los que depende estén compilados.

La herramienta compila **de a un módulo por vez**. En cada paso elige el próximo entre los módulos que están **listos**. Un módulo está listo cuando todavía no se compiló y todos los módulos de los que depende ya se compilaron.

Cuando hay más de un módulo listo, la herramienta elige así:

1. El módulo de **menor prioridad**.
2. Si varios módulos listos comparten esa prioridad mínima, el de **menor número**.

Los números de módulo no se repiten, así que esas dos reglas siempre eligen un único módulo. El orden de compilación queda determinado por completo.

Si las dependencias forman un ciclo, quedan módulos que nunca llegan a estar listos y la compilación no se puede completar.

Determine el **orden en que la herramienta compila los módulos**.

Entrada

- La primera línea contiene dos enteros V y A ($1 \leq V \leq 5 \times 10^5$, $0 \leq A \leq 10^6$), la cantidad de módulos y la cantidad de dependencias.

- La segunda línea contiene V enteros $P_1, P_2, \dots, P_V$ ($1 \leq P_i \leq 10^9$), donde P_i es la prioridad del módulo i .
- Las siguientes A líneas contienen dos enteros u y v ($1 \leq u, v \leq V$, $u \neq v$), que indican que el módulo u debe compilarse **antes** que el módulo v .

Las prioridades pueden repetirse. Las dependencias no se repiten.

Salida

Si se pueden compilar todos los módulos, imprima V líneas. Cada línea contiene el número de un módulo, en el orden en que la herramienta lo compila.

Si las dependencias forman un ciclo, imprima una única línea con la palabra `imposible`, y nada más. En ese caso no se imprime ningún módulo, ni siquiera los que sí se podrían haber compilado.

Restricciones

- Utilizar un **heap binario** para decidir cuál es el próximo módulo a compilar.
- Resolver en orden temporal $O((V+A)\log V)$ en el peor caso, siendo V la cantidad de módulos y A la cantidad de dependencias.
- Orden espacial $O(V+A)$.

Ejemplo

Input 1

```
5 4
5 2 1 4 3
1 3
2 3
3 4
3 5
```

Output 1

```
2
1
3
5
4
```

Explicación 1

Las prioridades son:

Módulo	1	2	3	4	5
Prioridad	5	2	1	4	3

El módulo 3 depende del 1 y del 2. Los módulos 4 y 5 dependen del 3.

En el diagrama, cada flecha va del módulo que se compila **antes** al que se compila **después**.

```
{"directed": true, "layout": "breadthfirst", "nodes": ["1","2","3","4","5"], "edges": [{"source": "1", "target": "3"}, {"source": "2", "target": "3"}, {"source": "3", "target": "4"}, {"source": "3", "target": "5"}]}
```

Paso	Módulos listos	Se compila	Motivo
1	1, 2	2	El módulo 2 tiene prioridad 2 y el módulo 1 tiene prioridad 5.
2	1	1	Es el único listo. El módulo 3 todavía espera al 1.
3	3	3	Ya se compilaron el 1 y el 2, sus dos dependencias.
4	4, 5	5	El módulo 5 tiene prioridad 3 y el módulo 4 tiene prioridad 4.
5	4	4	Es el único que queda.

Notar que el orden **no** es 1, 2, 3, 4, 5. Ese también respeta las dependencias, pero ignora las prioridades: en el paso 1 elegiría el módulo 1, que tiene prioridad 5, cuando el módulo 2 estaba listo con prioridad 2.

Input 2

```
4 2
7 7 7 7
3 1
4 2
```

Output 2

```
3
1
4
2
```

Explicación 2

Los cuatro módulos tienen la misma prioridad, así que decide siempre el número de módulo.

Las dependencias forman dos cadenas independientes: el 3 antes que el 1, y el 4 antes que el 2.

```
{"directed": true, "layout": "breadthfirst", "nodes": ["1","2","3","4"], "edges": [{"source": "1", "target": "3"}, {"source": "2", "target": "3"}, {"source": "3", "target": "4"}]}
```

Paso	Módulos listos	Se compila	Motivo
1	3, 4	3	Empatan en prioridad. El módulo 3 tiene número menor.
2	1, 4	1	Empatan en prioridad. El módulo 1 tiene número menor.
3	4	4	Es el único listo.
4	2	2	Quedó listo recién cuando se compiló el 4.

En el paso 2 el módulo 1 ya está listo, porque su única dependencia, el módulo 3, se compiló en el paso 1.

El resultado no es 1, 2, 3, 4. Aunque todos empaten en prioridad, el módulo 1 no puede ir primero: depende del módulo 3.

Input 3

```
4 0
3 1 3 2
```

Output 3

```
2
4
1
3
```

Explicación 3

No hay ninguna dependencia, así que los cuatro módulos están listos desde el principio y se compilan por prioridad.

Módulo	1	2	3	4
Prioridad	3	1	3	2

El módulo 2 tiene la menor prioridad y va primero. Después el módulo 4. Los módulos 1 y 3 empatan con prioridad 3, así que decide el número: primero el 1 y después el 3.

Input 4

```
3 3
1 2 3
1 2
2 3
3 1
```

Output 4

```
imposible
```

Explicación 4

El módulo 1 debe compilarse antes que el 2, el 2 antes que el 3, y el 3 antes que el 1. Las tres dependencias forman un ciclo.

```
{"directed": true, "layout": "circle", "nodes": ["1","2","3"], "edges": [["1","2"
```

Ningún módulo queda listo en el primer paso, porque los tres esperan a otro. La compilación no puede empezar.

Input 5

```
5 4
1 2 3 4 5
1 2
3 4
4 5
5 3
```

Output 5

```
imposible
```

Explicación 5

Los módulos 3, 4 y 5 forman un ciclo entre ellos. Los módulos 1 y 2 no participan del ciclo y sí se podrían compilar: el 1 está listo desde el principio, y el 2 queda listo apenas se compila el 1.

```
{"directed": true, "layout": "cose", "nodes": ["1","2","3","4","5"], "edges": [["1"
```

Igual la respuesta es `imposible`, y **no se imprime el 1 ni el 2**. Alcanza con que un solo módulo no se pueda compilar para que la compilación completa sea imposible.

Ejercicio 5 - Mantenimiento de senderos

Descripción

Un parque nacional tiene V refugios de montaña, numerados de 1 a V , unidos por senderos. Cada sendero conecta dos refugios y se puede recorrer en los dos sentidos.

Mantener un sendero abierto cuesta dinero, y cada sendero tiene su propio costo de mantenimiento. Los senderos que el parque no mantiene se cierran y no se pueden usar.

La administración necesita que desde cualquier refugio se pueda llegar a cualquier otro usando solo senderos mantenidos, tal vez pasando por refugios intermedios. Dentro de esa condición, quiere gastar lo menos posible.

Determine el **costo total mínimo de mantenimiento**.

Si ni siquiera manteniendo todos los senderos del parque se puede llegar de cualquier refugio a cualquier otro, la tarea es imposible.

Entrada

- La primera línea contiene dos enteros V y E ($1 \leq V \leq 10^5$, $0 \leq E \leq 10^6$), la cantidad de refugios y la cantidad de senderos.
- Las siguientes E líneas contienen tres enteros u , v y w ($1 \leq u, v \leq V$, $u \neq v$, $1 \leq w \leq 10^6$), que indican que hay un sendero entre el refugio u y el refugio v cuyo mantenimiento cuesta w .

Puede haber más de un sendero entre el mismo par de refugios, con costos distintos.

Salida

Imprima un único entero: el costo total mínimo de mantenimiento.

Si hay un solo refugio, no hace falta mantener ningún sendero e imprima `0`.

Si no es posible conectar todos los refugios entre sí, imprima `imposible`.

Restricciones

- Utilizar el algoritmo de **Kruskal** con una estructura de **conjuntos disjuntos**.
- La estructura de conjuntos disjuntos debe implementar las **dos optimizaciones vistas en el curso**:
 - **Compresión de caminos** al buscar el representante de un refugio.
 - **Unión por altura**, colgando siempre el árbol más bajo del más alto.
- Resolver en orden temporal $O(E \log E)$ en el peor caso, siendo E la cantidad de senderos.
- Orden espacial $O(V + E)$.
- El costo total puede superar el rango de un entero de 32 bits.

Ejemplo

Input 1

```
5 6
1 2 3
2 3 4
1 3 10
3 4 2
4 5 7
3 5 8
```

Output 1

16

Explicación 1

```
{"nodes": ["1", "2", "3", "4", "5"], "edges": [{"1", "2", 3}, {"2", "3", 4}, {"1", "3", 10}, [
```

Los senderos, mirados de más barato a más caro:

Sendero	Costo	¿Se mantiene?	Motivo
3 – 4	2	sí	Conecta dos refugios que estaban separados.
1 – 2	3	sí	Conecta dos refugios que estaban separados.
2 – 3	4	sí	Une el grupo 1, 2 con el grupo 3, 4.
4 – 5	7	sí	Trae el refugio 5, el último que faltaba.
3 – 5	8	no	Los refugios 3 y 5 ya se alcanzan entre sí.
1 – 3	10	no	Los refugios 1 y 3 ya se alcanzan entre sí.

El costo total es $2+3+4+7=16$.

Mantener los seis senderos costaría 34, pero dos de ellos no agregan ningún recorrido nuevo. Con cuatro senderos alcanza para conectar cinco refugios.

Input 2

```
4 5
1 2 1
2 3 6
3 4 6
4 1 6
2 4 6
```

Output 2

```
13
```

Explicación 2

```
{"nodes": ["1", "2", "3", "4"], "edges": [["1", "2", 1], ["2", "3", 6], ["3", "4", 6], ["4", "1", 6], ["2", "4", 6]]}
```

Para conectar cuatro refugios hacen falta tres senderos. El más barato del parque cuesta 1, y los otros cuatro cuestan 6 cada uno. Hay que mantener el de 1 y dos de los de 6, así que el costo total es $1+6+6=13$.

Este ejemplo muestra que **el conjunto elegido no es único, pero el costo sí lo es**. Hay cinco formas distintas de conectar los cuatro refugios con tres senderos:

- 1 - 2 , 2 - 3 , 3 - 4
- 1 - 2 , 2 - 3 , 4 - 1
- 1 - 2 , 2 - 3 , 2 - 4
- 1 - 2 , 3 - 4 , 4 - 1
- 1 - 2 , 3 - 4 , 2 - 4

Las cinco cuestan 13. La única combinación de tres senderos que no sirve es 1 - 2 , 4 - 1 , 2 - 4 , porque conecta los refugios 1, 2 y 4 entre sí pero deja al refugio 3 afuera.

Input 3

```
5 3
1 2 4
2 3 5
4 5 6
```

Output 3

```
imposible
```

Explicación 3

```
{"nodes": ["1", "2", "3", "4", "5"], "edges": [["1", "2", 4], ["2", "3", 5], ["4", "5", 6]]}
```

Manteniendo los tres senderos, los refugios quedan agrupados así: 1, 2, 3 por un lado y 4, 5 por el otro.

No hay ningún sendero entre esos dos grupos, así que no existe forma de ir del refugio 1 al refugio 4. La respuesta es imposible.

Notar que sí se logra conectar parte del parque. Eso no alcanza: hay que poder ir de **cualquier** refugio a **cualquier** otro.

Input 4

```
1 0
```

Output 4

```
0
```

Explicación 4

Hay un solo refugio y ningún sendero. Desde ese refugio ya se llega a todos los refugios del parque, porque es el único.

No hace falta mantener ningún sendero, así que el costo total es 0, y no imposible.

Input 5

```
4 5
1 2 1
2 3 1
1 3 1
3 4 100
3 4 50
```

Output 5

```
52
```

Explicación 5

```
{"nodes": ["1", "2", "3", "4"], "edges": [["1", "2", 1], ["2", "3", 1], ["1", "3", 1], ["3", "4", 50], ["3", "4", 100]}
```

Los refugios 1, 2 y 3 están unidos por un triángulo de senderos que cuestan 1 cada uno. Los tres son baratos, pero **mantener los tres sería un desperdicio**: con dos cualesquiera de ellos los tres refugios ya se alcanzan entre sí. El tercero solo repite un recorrido que ya existe.

Sendero	Costo	¿Se mantiene?	Motivo
1 – 2	1	sí	Conecta dos refugios que estaban separados.
2 – 3	1	sí	Trae el refugio 3.
1 – 3	1	no	Los refugios 1 y 3 ya se alcanzan por el refugio 2.
3 – 4	50	sí	Es la única forma de traer el refugio 4, y la más barata de las dos.
3 – 4	100	no	El refugio 4 ya está conectado.

El costo total es $1+1+50=52$.

Quedarse con los tres senderos más baratos del parque daría 3, pero esa respuesta es incorrecta: el refugio 4 quedaría aislado. No alcanza con elegir los más baratos, hay que descartar los que no conectan nada nuevo.

Este ejemplo también muestra que puede haber dos senderos entre el mismo par de refugios. Entre el 3 y el 4 hay uno de 50 y otro de 100, y se mantiene el de 50.

Formato de entrega (código fuente)

El formato de entrega del obligatorio va a constar de un archivo zip que en su interior contenga únicamente archivos con extensiones ".cpp" y ".h" en caso de C/C++ o ".java" en caso de utilizar Java, además de la **bitácora** (`bitacora.md`), la **declaración de autoría** (`autoria.md`) y la **justificación de órdenes** (`justificacion.md`) en la raíz. También podrán existir directorios para organizar mejor el código y poder reutilizar implementaciones en varios ejercicios.

Es importante que el archivo zip no contenga archivos de ningún otro tipo, como test/pruebas, archivos binarios, etc. Las únicas excepciones son la bitácora (bitacora.md), la declaración de autoría (autoria.md) y la justificación de órdenes (justificacion.md).

Recomendamos basarse en los templates provistos por la cátedra para minimizar la posibilidad de errores en la entrega. Los templates ya incluyen `bitacora.md`, `autoria.md` y `justificacion.md` con la estructura esperada.

- Obligatorio 1 (ejercicios 1 a 5): [Template C++](#) | [Template Java](#)
- Obligatorio 2 (ejercicios 6 a 10): [Template C++](#) | [Template Java](#)

C/C++

En el caso de C/C++, la estructura dentro del zip tendrá el siguiente formato:

```
|— directorioA
|— directorioB
...
|— directorioZ
|— ejercicio1.cpp
|— ejercicio2.cpp
...
|— ejercicio10.cpp
```

Por ejemplo, si tuviéramos 10 ejercicios, un posible zip a entregar podría tener:

```
|— funciones
|  |— enteros.cpp
|— tads
|  |— ListImp.cpp
|  |— List.h
|— ejercicio1.cpp
|— ejercicio2.cpp
...
|— ejercicio10.cpp
```

Notar que es obligatorio que en la raíz del zip se encuentren los archivos "ejercicioX.cpp". Los directorios para funciones auxiliares y TADs pueden llamarse de cualquier forma y no son obligatorios. Un ejemplo donde no se utilizan directorios auxiliares podría ser:

```
├─ enteros.cpp
├─ ListImp.cpp
├─ List.h
├─ ejercicio1.cpp
├─ ejercicio2.cpp
...
└─ ejercicio10.cpp
```

Cómo compilar

La forma de compilar cada ejercicio es la vista en el curso, pasando por parámetro el archivo raíz "ejercicioX.cpp", siendo X el número de ejercicio, e indicando en el parámetro "-o" el nombre del archivo compilado:

```
g++ -o ejercicioX ejercicioX.cpp -std=c++11
```

Cómo realizar las inclusiones

Las inclusiones parten del archivo "raíz" del programa, es decir, el "ejercicioX.cpp".

Dependiendo de si los archivos a incluir se encuentran en el mismo directorio o no, es cómo se realizará la inclusión.

Con directorios

Dado el siguiente caso:

```
├─ funciones
│   └─ enteros.cpp
├─ tads
│   ├── ListImp.cpp
│   └─ List.h
...
```

Se definen las inclusiones de la siguiente manera:

```
#include "tads/List.h"
#include "tads/ListImp.cpp"
#include "funciones/enteros.cpp"
```

Sin directorios

Dado el caso en el que no se usan directorios:

```
|— enteros.cpp
|— ListImp.cpp
|— List.h
...
```

Se definen las inclusiones de la siguiente manera:

```
#include "List.h"
#include "ListImp.cpp"
#include "enteros.cpp"
```

Aclaración sobre TADs

Notar que en el archivo raíz se incluye tanto la especificación del TAD (List.h) como la implementación (ListImp.cpp). Lo correcto es incluir únicamente la especificación y asignar la implementación al momento de compilar. Esto resulta en problemas para realizar debugging y en que la compilación sea más compleja de estandarizar, por lo que se decide que es aceptable realizar la inclusión de la implementación.

JAVA

En el caso de Java, la estructura dentro del zip tendrá el siguiente formato:

```
|— directorioA
|— directorioB
...
|— directorioZ
|— Ejercicio1.java
|— Ejercicio2.java
...
|— Ejercicio10.java
```

Por ejemplo, si tuviéramos 10 ejercicios, un posible zip a entregar podría tener:

```
|— funciones
|  |— Enteros.java
|— tads
|  |— List.java
|— Ejercicio1.java
|— Ejercicio2.java
...
|— Ejercicio10.java
```

Notar que es obligatorio que en la raíz del zip se encuentren los archivos "EjercicioX.java". Los directorios para funciones auxiliares y TADs pueden llamarse de cualquier forma y no son obligatorios. Un ejemplo donde no se utilizan directorios auxiliares podría ser:

```
|— Enteros.java
|— List.java
|— Ejercicio1.java
|— Ejercicio2.java
...
|— Ejercicio10.java
```

Cómo compilar

La forma de compilar cada ejercicio es la vista en el curso, pasando por parámetro el archivo raíz "EjercicioX.java", siendo X el número de ejercicio:

```
javac EjercicioX.java
```

Cómo realizar las importaciones

Las importaciones parten del archivo "raíz" del programa, es decir, el "EjercicioX.java".

Dependiendo de si los archivos a incluir se encuentran en el mismo directorio o no, es cómo se realizan.

Con directorios

Dado el siguiente caso:

```
|— funciones
|   |— Enteros.java
|— tads
|   |— List.java
..
```

Se definen las importaciones de la siguiente manera:

```
import tads.List;
import funciones.Enteros;
```

Para que esto funcione es importante definir los paquetes java de las clases que se encuentran en esos directorios.

Dado el ejemplo anterior, la clase Enteros.java debe definir su paquete al comienzo del archivo:

```
package funciones;
```

De la misma forma lo debe hacer la clase List.java:

```
package tads;
```

Las clases de la raíz, "EjercicioX.java" por ejemplo, no deben tener definido un paquete.

Sin directorios

Dado el caso en el que no se usan directorios:

```
├─ Enteros.java  
├─ List.java  
...
```

No es necesario definir ninguna importación. Es importante que ninguna clase del directorio raíz tenga definido un paquete al comienzo de la misma.

Bitácora

La documentación del obligatorio consiste en una **bitácora de trabajo** en formato **markdown** (`bitacora.md`), que se entrega junto con el código fuente.

La bitácora es un registro cronológico del proceso de trabajo: algo similar al historial de git, pero más extenso y narrado. Lo importante no es el resultado final sino el **proceso**: hilos de pensamiento, decisiones, intentos descartados, correcciones y avances.

Qué debe contener

- **Una entrada por cada día trabajado**, con la fecha.
- **Quién trabajó** ese día: un integrante en particular o en conjunto.
- **Qué se hizo y cómo se pensó**: ideas exploradas, decisiones de diseño y su justificación, partes de implementaciones, bugs encontrados y cómo se corrigieron, resultados de pruebas, dudas que quedaron abiertas.
- **Uso de IA**, si lo hubo ese día: herramienta, consultas relevantes y qué se hizo con la respuesta. Ver **Uso de IA**.

No hay un mínimo de largo por entrada: un día de poco avance puede tener dos líneas. Lo que se evalúa es que la bitácora refleje fielmente el proceso real de trabajo.

Ejemplo

```
# Bitácora – Obligatorio 1

## 2026-09-02 – Juana
- Leí la letra del ejercicio 3. Primera idea: mantener los elementos ordenados en un vector, pero la inserción queda  $O(n)$  y la letra pide  $O(\log n)$ .
- Pensándolo mejor, un heap resuelve la inserción y el mínimo en  $O(\log n)$ . Mañana lo hablamos con Pedro.

## 2026-09-04 – En conjunto (Juana y Pedro)
- Decidimos usar un min-heap para el ejercicio 3. Implementamos el TAD Heap con arreglo (flotar/hundir).
- Bug: `hundir` comparaba solo con el hijo izquierdo; en el caso de prueba 4 devolvía un orden incorrecto. Lo corregimos comparando con el menor de los dos hijos.
- Pasan los casos de prueba 1 a 5 del ejercicio 3.

## 2026-09-06 – Pedro
- Empecé el ejercicio 4. Le pregunté a ChatGPT qué significa el error "segmentation fault" y estrategias para encontrar el origen; con eso encontré un puntero sin inicializar en `ListImp.cpp` (queda citado acá y comentado en el código).
- El ejercicio 4 compila pero todavía no pasa el caso de prueba 2.
```

Formato de entrega

- Archivo `bitacora.md` en la **raíz del zip** de la entrega, junto al código fuente, la **declaración de autoría** (`autoria.md`) y la **justificación de órdenes** (`justificacion.md`).
- Debe incluir una carátula (título con los datos de los estudiantes al inicio del archivo).

Importante: La falta de bitácora, o una bitácora que no refleje el proceso de trabajo (por ejemplo, escrita íntegramente el día de la entrega), implicará la pérdida de puntos.

Autoría

Los estudiantes deben especificar las fuentes de información y recursos utilizados para la resolución de cada ejercicio. Esto incluye:

- Código discutido o desarrollado en clase.
- Recursos o ejemplos encontrados en la web. Se debe especificar la URL de la página.
- Código generado por herramientas de inteligencia artificial generativa (IAG). Ver las pautas en **Uso de IA**.

Importante: Es imprescindible citar correctamente todas las fuentes utilizadas. La omisión de esta información podría considerarse como plagio, lo cual conllevaría a acciones disciplinarias según las políticas de la universidad. La transparencia en la citación de fuentes es crucial para mantener la integridad académica del obligatorio.

Cómo declarar la autoría

La declaración de autoría se entrega en un archivo `autoría.md` en la **raíz del zip**, junto al código fuente y la **bitácora**, con un apartado por ejercicio. Si un ejercicio no utilizó fuentes externas, también debe indicarse. Los templates provistos por la cátedra ya incluyen este archivo con la estructura esperada.

Ejemplo de declaración

```
# Declaración de autoría – Obligatorio 1

## Ejercicio 3
- TAD Heap basado en la implementación vista en clase (semana 8), adaptado para soportar elementos con prioridad repetida.
- Corrección de la función `hundir` con ayuda de ChatGPT (debugging sobre código propio, ver entrada del 2026-09-04).

## Ejercicio 4
- Función de lectura de grafos basada en el ejemplo de https://cp-algorithms.com/graph/breadth-first-search.html
- El resto de la implementación es íntegramente propia.

## Ejercicio 5
- Implementación íntegramente propia, sin fuentes externas.
```

Ejemplo en el código fuente

Además de la declaración en `autoría.md`, los fragmentos tomados de otras fuentes o generados por IAG deben señalarse con un comentario en el lugar donde se usan:

```
// Fragmento generado con GitHub Copilot: plantilla de la clase NodoAVL.  
// Adaptado: se agregó el campo altura y el constructor por parámetros.  
struct NodoAVL {  
    ...  
};
```

```
// Basado en el algoritmo de Floyd visto en clase (semana 10).  
void ordenarHeap(int* arr, int n) {  
    ...  
}
```

Consideraciones

- Citar una fuente **no exime** de comprender el código: en la **defensa** se debe poder explicar y justificar todo lo entregado, incluyendo los fragmentos de terceros o generados por IAG.
- En caso de existir dudas sobre la autoría, plagio o uso no atribuido de IAG, el docente podrá convocar al equipo a una instancia específica e individual sobre el tema.

Defensa

Las defensas son **presenciales, escritas y obligatorias**. La no realización de una defensa implica la pérdida de la totalidad de los puntos del obligatorio correspondiente.

La defensa es una instancia clave del proceso de evaluación, en la que el estudiante debe demostrar una comprensión clara sobre los algoritmos, estructuras de datos y decisiones técnicas empleadas en la elaboración de los obligatorios.

Instancias

Hay **dos defensas** durante el semestre:

Defensa	Cuándo	Contenido
Primera	En la clase siguiente a la entrega del obligatorio	Ejercicios del primer obligatorio
Segunda	Junto con el segundo parcial	Ejercicios del segundo obligatorio

Modalidad

- **Forma escrita:** cada defensa consiste en un conjunto de preguntas escritas, **una pregunta por ejercicio**.
- **Duración:** cada defensa tiene una duración máxima de **30 minutos**.
- **Presencial y obligatoria:** ambas defensas se realizan de forma presencial. No existen defensas orales.
- **Contenido:** las preguntas se centran en la justificación de algoritmos, estructuras de datos, versiones y decisiones específicas tomadas en cada uno de los ejercicios.

Puntaje

Importante: Los puntos de cada ejercicio están atados a la defensa: no poder contestar la pregunta correspondiente a un ejercicio implica la **pérdida de los puntos de dicho ejercicio**.

Expectativas

Durante cada defensa, el estudiante debe demostrar:

- Comprensión sólida sobre los algoritmos y estructuras utilizados.
- Capacidad de justificar decisiones técnicas y versiones elegidas.
- Claridad en la explicación sobre el funcionamiento general del código desarrollado.

El objetivo no es que el estudiante memorice su código, sino que sea capaz de explicar y justificar las decisiones técnicas adoptadas, así como demostrar su comprensión del funcionamiento de los algoritmos y estructuras empleadas.

Preparación

- Revisar previamente los algoritmos y estructuras empleadas en cada ejercicio, así como las razones detrás de cada elección técnica.
- La **bitácora** es una excelente herramienta de preparación: repasar las entradas del proceso de trabajo ayuda a reconstruir el razonamiento detrás de cada decisión.

RECORDATORIO: IMPORTANTE PARA LA ENTREGA

La entrega de los obligatorios será en formato digital online, a excepción de algunas materias que se entregarán en Bedelía y en ese caso recibirá información específica en el dictado de la misma.

Los principales aspectos a destacar sobre la **entrega online de obligatorios** son:

1. Ingresá al sistema de Gestión.
2. En el menú, seleccioná el ítem “Evaluaciones” y la instancia de evaluación correspondiente, que figura bajo el título “Inscripto”.
3. Para iniciar la entrega hacé clic en el ícono correspondiente.
4. Ingresá el número de estudiante de cada uno de los integrantes y hacé clic en “Agregar”. El sistema confirmará que los integrantes estén inscriptos al obligatorio y, de ser así, mostrará el nombre y la fotografía de cada uno de ellos. Una vez agregados todos los integrantes, hacé clic en “Crear equipo”.

Cualquier integrante podrá:

- Modificar la integración del equipo.
 - Subir el archivo de la entrega.
5. Seleccioná el archivo que deseás entregar. Verificá el nombre del archivo que aparecerá en la pantalla y hacé clic en “Subir” para iniciar la entrega. Cada equipo (hasta 2 estudiantes) debe entregar **un único archivo en formato zip** (los documentos de texto deben ser pdf, y deben ir dentro del zip). El archivo a subir debe tener un **tamaño máximo de 40mb**.
 6. Cuando el archivo quede subido, se mostrará el nombre generado por el sistema, el tamaño y la fecha en que fue subido.
 7. El sistema enviará un e-mail a todos los integrantes del equipo informando los detalles del archivo entregado y confirmando que la entrega fue realizada correctamente.
 8. Podés cerrar la pestaña de entrega y continuar utilizando Gestión o salir del sistema.
 9. **La hora tope para subir el archivo será las 21:00** del día fijado para la entrega.
 10. La entrega se podrá realizar desde cualquier lugar (ej. hogar del estudiante, laboratorios de la Universidad, etc).
 11. Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con la Coordinadora de Cursos o Coordinación Adjunta antes de las 20:00h del día de la entrega, a través del mail: adjuntos_ei@ort.edu.uy.